

Proyecto Base de Datos 1.º DAW

Duración estimada: 15–20 h

ESTIMACIÓN DE TIEMPO

- Diseño E-R: 3–4 h
- Modelo relacional: 2–3 h
- DDL: 2–3 h
- DML: 2 h
- Consultas: 3–4 h
- PL/SQL: 3–4 h

ENTREGABLES

1. Documento PDF con:
 - Modelo E-R
 - Modelo relacional
 - Explicación del diseño
2. Script SQL con:
 - DDL
 - DML
 - Consultas
 - PL/SQL
3. (Opcional) Capturas de ejecución
4. Se comprime todo en un ZIP.

PROYECTO 1: Sistema de Gestión de Clínica Veterinaria

ENUNCIADO

Una clínica veterinaria desea informatizar la gestión de su negocio. Actualmente trabajan con registros en papel y hojas de cálculo, lo que genera errores y dificulta el acceso a la información.

La clínica necesita almacenar información sobre:

- Clientes (propietarios de mascotas)
- Mascotas
- Veterinarios
- Citas
- Consultas médicas
- Tratamientos y medicamentos
- Facturación

El sistema debe permitir gestionar todo el ciclo: desde que un cliente pide cita, hasta que se realiza la consulta, se prescribe tratamiento y se genera una factura.

REQUERIMIENTOS DEL PROYECTO

1. Modelo Entidad-Relación (E-R)

Diseñar el modelo E-R que incluya:

Entidades mínimas:

- Cliente
- Mascota
- Veterinario
- Cita
- Consulta
- Tratamiento
- Medicamento
- Factura

Requisitos:

- Un cliente puede tener varias mascotas
- Cada mascota pertenece a un único cliente

- Una mascota puede tener múltiples citas
- Cada cita es atendida por un veterinario
- Una cita puede generar una consulta
- En una consulta se pueden recetar varios tratamientos
- Cada tratamiento puede incluir varios medicamentos
- Cada consulta genera una factura

Incluir:

- Claves primarias
- Cardinalidades
- Especialización (total/parcial)

2. Modelo Relacional

Transformar el modelo E-R al modelo relacional:

Requisitos:

- Tablas con claves primarias y foráneas
- Resolver relaciones N:M mediante tablas intermedias
- Definir correctamente tipos de datos

3. DDL (Data Definition Language)

Crear el script SQL de creación de la base de datos:

Incluir:

- `CREATE TABLE`
- Claves primarias y foráneas
- Restricciones:
 - `NOT NULL`
 - `UNIQUE`
 - `CHECK`
- Índices (al menos 2)

4. DML (Data Manipulation Language)

Insertar datos de prueba:

Requisitos:

- Al menos:
 - 10 clientes
 - 15 mascotas

- 5 veterinarios
- 20 citas
- 20 consultas
- 10 tratamientos
- 10 medicamentos
- 20 facturas

5. Consultas SQL

♦ Básicas:

1. Listar todos los clientes
2. Mostrar mascotas de un cliente
3. Mostrar citas de un veterinario

♦ Intermedias:

4. Número de mascotas por cliente
5. Total facturado por cliente
6. Veterinario con más citas

♦ Avanzadas (con subconsultas):

7. Clientes que tienen más mascotas que el promedio
8. Mascotas que han tenido más de 3 consultas
9. Veterinarios que no han tenido citas
10. Facturas por encima del promedio

♦ Con JOIN:

11. Mostrar cliente + mascota + consulta
12. Mostrar tratamientos con medicamentos asociados

6. Subconsultas

Incluir al menos 5 subconsultas que usen:

- **IN EXISTS ANY / ALL**
- Subconsultas correlacionadas
- Obtener los clientes que tienen mascotas que han tenido al menos una consulta.
- Listar los veterinarios que han atendido al menos una cita.
- Mostrar las facturas cuyo importe es mayor que cualquiera de las facturas de un cliente específico.
- Obtener los clientes cuyo total facturado es mayor que el de todos los demás clientes.

- Mostrar las mascotas que tienen más consultas que el promedio de consultas de su propio cliente.

7. PL/SQL

♦ Procedimientos (mínimo 2):

Ejemplos:

- Registrar una nueva cita
- Generar automáticamente una factura

♦ Funciones (mínimo 2):

Ejemplos:

- Calcular total gastado por cliente
- Contar número de consultas de una mascota

♦ Triggers (mínimo 1):

Ejemplos:

- Generar factura automáticamente al insertar consulta
- Evitar citas en fechas pasadas

8. Extras (opcional para subir nota)

- Vistas (**CREATE VIEW**)
- Control de errores en PL/SQL
- Uso de secuencias (**SEQUENCE**)
- Procedimientos con parámetros

PROYECTO:Sistema de Gestión de Biblioteca

ENUNCIADO

Una red de bibliotecas públicas necesita un sistema para gestionar préstamos, usuarios y catálogo de libros. Actualmente tienen problemas para controlar devoluciones, disponibilidad de ejemplares y multas por retrasos.

El sistema debe permitir:

- Registrar usuarios
- Gestionar libros y ejemplares físicos
- Controlar préstamos y devoluciones
- Aplicar sanciones por retrasos
- Gestionar reservas de libros

REQUERIMIENTOS

1. Modelo Entidad-Relación (E-R)

Entidades y/o relaciones:

- Usuario
- Libro
- Autor
- Ejemplar
- Préstamo
- Reserva
- Multa

Reglas del EER:

- Un libro puede tener varios autores (N:M)
- Un libro tiene varios ejemplares
- Un usuario puede tener varios préstamos
- Un ejemplar solo puede estar prestado a un usuario a la vez
- Un usuario puede reservar libros
- Un préstamo puede generar una multa

Incluir:

- Claves primarias
- Cardinalidades

- Especialización (total/parcial)

2. Modelo Relacional

Transformar el modelo E-R al modelo relacional:

Requisitos:

- Tablas con claves primarias y foráneas
- Resolver relaciones N:M mediante tablas intermedias
- Definir correctamente tipos de datos

3. DDL (Data Definition Language)

Crear el script SQL de creación de la base de datos:

Incluir:

- `CREATE TABLE`
- Claves primarias y foráneas
- Restricciones:
 - `NOT NULL`
 - `UNIQUE`
 - `CHECK`
- Índices (al menos 2)

4. DML

Insertar:

- 15 usuarios
- 20 libros
- 30 ejemplares
- 10 autores
- 25 préstamos
- 10 reservas
- 10 multas

5. Consultas SQL

♦ Básicas:

1. Listar libros disponibles
2. Mostrar préstamos de un usuario
3. Mostrar reservas activas

♦ **Intermedias:**

4. Número de préstamos por usuario
5. Libros más prestados
6. Total de multas por usuario

♦ **Avanzadas (subconsultas):**

7. Usuarios con más préstamos que la media
8. Libros nunca prestados
9. Usuarios con multas pendientes
10. Ejemplares no reservados

♦ **JOIN:**

11. Libro + autor
12. Usuario + préstamo + multa

6. Subconsultas

Al menos 5 usando:

- **IN EXISTS ANY / ALL**
- Listar los usuarios que han realizado préstamos de libros concretos (por ejemplo, de una lista de libros populares).
- Mostrar los libros que tienen al menos un ejemplar prestado actualmente.
- Obtener los usuarios que tienen más préstamos que cualquiera de los usuarios con multas.
- Mostrar los libros que han sido prestados más veces que todos los demás libros.
- Listar los usuarios cuyo número de préstamos es superior a la media de préstamos de usuarios de su misma situación (por ejemplo, con o sin multas).

7. PL/SQL

Procedimientos:

- **Registrar préstamo**
- **Registrar devolución (calculando multa)**

Funciones:

- Calcular multas de un usuario
- Contar préstamos activos

Investigación Trigger:

- Generar multa automáticamente si hay retraso

8. Extras

- Vistas: libros disponibles
- Secuencias para IDs
- Control de errores

PROYECTO 2: Sistema de Gestión de Citas y Reservas de Recursos

ENUNCIADO

Una empresa dispone de recursos compartidos (salas, equipos, vehículos) que deben ser reservados por empleados. Actualmente existen conflictos por solapamiento de reservas y falta de control.

El sistema debe:

- Gestionar empleados
- Registrar recursos disponibles
- Permitir reservas
- Evitar conflictos de horarios
- Controlar uso y disponibilidad

REQUERIMIENTOS

1. Modelo E-R

Entidades y/o relaciones:

- Empleado
- Recurso
- TipoRecurso
- Reserva
- Ubicación

Reglas:

- Un recurso pertenece a un tipo
- Un recurso está en una ubicación
- Un empleado puede hacer varias reservas
- Un recurso no puede tener reservas solapadas
- Una reserva tiene fecha inicio y fin

2. Modelo Relacional

Transformar el modelo E-R al modelo relacional:

Requisitos:

- Tablas con claves primarias y foráneas
- Resolver relaciones N:M mediante tablas intermedias
- Definir correctamente tipos de datos

3. DDL (Data Definition Language)

Crear el script SQL de creación de la base de datos:

Incluir:

- `CREATE TABLE`
- Claves primarias y foráneas
- Restricciones:
 - `NOT NULL`
 - `UNIQUE`
 - `CHECK`
- Índices (al menos 2)

4. DML

Insertar:

- 10 empleados
- 15 recursos
- 5 tipos de recurso
- 5 ubicaciones
- 30 reservas

5. Consultas SQL

♦ Básicas:

1. Listar recursos disponibles
2. Ver reservas de un empleado
3. Mostrar recursos por tipo

♦ Intermedias:

4. Recursos más utilizados
5. Número de reservas por empleado
6. Recursos nunca reservados

♦ Avanzadas (subconsultas):

7. Empleados con más reservas que la media

8. Recursos con reservas en una fecha concreta
9. Recursos sin uso en el último mes
10. Reservas que se solapan (detección)

♦ **JOIN:**

11. Empleado + reserva + recurso
12. Recurso + ubicación + tipo

6. Subconsultas

- **IN EXISTS ANY / ALL**
- Obtener los empleados que han reservado recursos de un tipo específico.
- Listar los recursos que tienen al menos una reserva activa.
- Mostrar los empleados con más reservas que cualquiera de los empleados de un departamento específico (si lo modelas).
- Obtener los recursos más utilizados, es decir, aquellos con más reservas que todos los demás recursos.
- Mostrar los recursos cuya cantidad de reservas es superior a la media de reservas de su mismo tipo.

7. PL/SQL

Procedimientos:

- Crear reserva validando disponibilidad
- Cancelar reserva

Funciones:

- Verificar disponibilidad de recurso
- Contar reservas activas

Investigación: Trigger

- Evitar inserción de reservas solapadas

8. Extras

- Vista: recursos disponibles por fecha
- Auditoría de reservas
- Secuencias

PROYECTO 3: Sistema de Gestión Académica Universitaria

ENUNCIADO

Una universidad necesita una base de datos para gestionar su actividad académica. Actualmente tienen sistemas separados para alumnos, profesores y asignaturas, lo que dificulta el seguimiento del rendimiento y la planificación.

El sistema debe permitir:

- Gestionar estudiantes y profesores
- Administrar asignaturas y titulaciones
- Controlar matrículas
- Registrar calificaciones
- Consultar rendimiento académico

REQUERIMIENTOS

Entidades y/o relaciones:

Entidades:

- Estudiante
- Profesor
- Asignatura
- Titulación
- Matrícula
- Calificación

Reglas:

- Un estudiante puede matricularse en varias asignaturas
- Una asignatura puede tener varios profesores
- Un profesor puede impartir varias asignaturas
- Una matrícula relaciona estudiante + asignatura + curso académico
- Cada matrícula tiene una calificación

Incluir:

- Relación N:M (Profesor–Asignatura)
- Relación N:M (Estudiante–Asignatura mediante Matrícula)
- Cardinalidades y claves

2. Modelo Relacional

Transformar el modelo E-R al modelo relacional:

Requisitos:

- Tablas con claves primarias y foráneas
- Resolver relaciones N:M mediante tablas intermedias
- Definir correctamente tipos de datos

3. DDL (Data Definition Language)

Crear el script SQL de creación de la base de datos:

Incluir:

- `CREATE TABLE`
- Claves primarias y foráneas
- Restricciones:
 - `NOT NULL`
 - `UNIQUE`
 - `CHECK`
- Índices (al menos 2)

4. DML

Insertar:

- 20 estudiantes
- 10 profesores
- 15 asignaturas
- 5 titulaciones
- 40 matrículas
- 40 calificaciones

5. Consultas SQL

♦ Básicas:

1. Listar estudiantes
2. Mostrar asignaturas de un estudiante
3. Profesores por asignatura

♦ **Intermedias:**

1. Nota media por estudiante
2. Número de alumnos por asignatura
3. Profesor con más asignaturas

♦ **Avanzadas:**

1. Estudiantes con nota media superior a la media global
2. Asignaturas sin alumnos
3. Profesores sin asignaturas
4. Estudiantes con todas las asignaturas aprobadas

♦ **JOIN:**

1. Estudiante + asignatura + nota
2. Profesor + asignatura

6. Subconsultas

- **IN EXISTS ANY / ALL NOT**
- Listar los estudiantes matriculados en asignaturas de una titulación específica.
- Mostrar las asignaturas que tienen al menos un estudiante matriculado.
- Obtener los estudiantes cuya nota media es mayor que la de cualquiera de los estudiantes de una titulación concreta.
- Mostrar los estudiantes cuya nota media es superior a la de todos los demás estudiantes.
- Listar los estudiantes cuya nota media es superior a la media de su propia titulación.

7. PL/SQL

Procedimientos:

- Matricular estudiante
- Registrar calificación

Funciones:

- Calcular nota media
- Contar asignaturas aprobadas

Investigación Trigger:

Evitar calificaciones fuera de rango (0–10)

8. Extras

Vista: expediente académico

Control de convocatorias

Secuencias

PROYECTO 4: Sistema de Gestión de Tienda Online (E-commerce)

ENUNCIADO

Una tienda online necesita gestionar su catálogo, clientes y pedidos. Actualmente tienen problemas con el control de stock, pedidos incompletos y seguimiento de ventas.

- El sistema debe:
- Gestionar clientes
- Administrar productos y categorías
- Registrar pedidos
- Controlar stock
- Gestionar pagos

REQUERIMIENTOS

1. Modelo E-R

Entidades y/o relaciones:

- Cliente
- Producto
- Categoría
- Pedido
- DetallePedido
- Pago

Reglas:

- Un cliente puede hacer varios pedidos
- Un pedido tiene varios productos (N:M → DetallePedido)
- Un producto pertenece a una categoría
- Un pedido tiene un pago asociado
- El stock debe actualizarse con cada pedido

2. Modelo Relacional

Transformar el modelo E-R al modelo relacional:

Requisitos:

- Tablas con claves primarias y foráneas

- Resolver relaciones N:M mediante tablas intermedias
- Definir correctamente tipos de datos

3. DDL (Data Definition Language)

Crear el script SQL de creación de la base de datos:

Incluir:

- `CREATE TABLE`
- Claves primarias y foráneas
- Restricciones:
 - `NOT NULL`
 - `UNIQUE`
 - `CHECK`
- Índices (al menos 2)

4. DML

Insertar:

- 15 clientes
- 20 productos
- 5 categorías
- 30 pedidos
- 60 detalles de pedido
- 30 pagos

5. Consultas SQL

♦ Básicas:

1. Listar productos
2. Ver pedidos de un cliente
3. Mostrar categorías

♦ Intermedias:

1. Total gastado por cliente
2. Productos más vendidos
3. Stock actual

♦ Avanzadas:

1. Clientes que han gastado más que la media
2. Productos nunca vendidos

3. Pedidos sin pago
4. Clientes sin pedidos

♦ **JOIN:**

1. Pedido + cliente + productos
2. Producto + categoría

6. Subconsultas

- **IN EXISTS ANY / ALL**
- Obtener los clientes que han realizado pedidos que incluyen ciertos productos.
- Listar los productos que han sido incluidos en al menos un pedido.
- Mostrar los clientes cuyo gasto total es mayor que el de cualquiera de los clientes sin pedidos recientes.
- Obtener los productos más vendidos (más vendidos que todos los demás productos).
- **(Correlacionada)**
Listar los clientes cuyo gasto total es superior al promedio de clientes de su misma categoría (si existe).

7. PL/SQL

Procedimientos:

- Crear pedido
- Registrar pago

Funciones:

- Calcular total de pedido
- Verificar stock

Investigación Trigger:

- Actualizar stock al insertar detalle de pedido

8. Extras

- Vista: pedidos completos
- Auditoría de stock
- Control de devoluciones